Clase 32 — Enrutamiento distance-vector: Bellman-Ford y el conteo a infinito

Bloque: D (Capa de red) · **Semana:** 11 · **Duración estimada:** 115 min **Correlativa:** Clase 31 (NAT, PAT y sus tricks) · **Sigue con:** Clase 33 (link-state, Dijkstra y OSPF) **Material de referencia:** Tanenbaum cap. 5 (§5.6) · RFC 2453, RFC 1058, RFC 2080

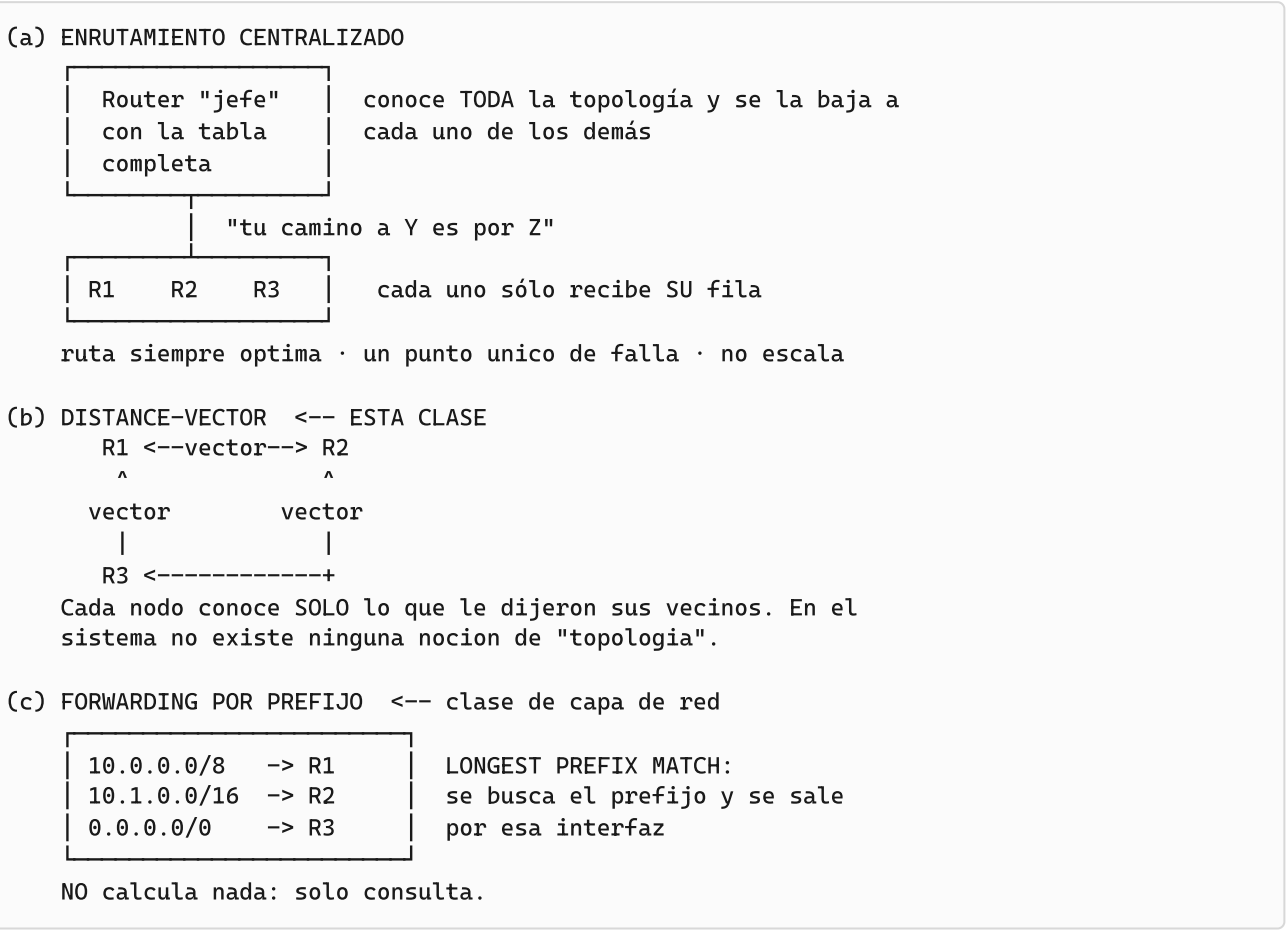
🎯 Al terminar esta clase vas a poder

1. Explicar qué intercambian dos routers **distance-vector** y por qué no conocen la topología.
2. Escribir la **ecuación de Bellman-Ford** y justificar por qué es correcta desde el **principio de optimalidad**.
3. Ejecutar **a mano, con la tabla de iteraciones**, el algoritmo DV hasta la convergencia.
4. Reproducir **el conteo a infinito** ronda por ronda y explicar **por qué** los routers se realimentan.
5. Describir las **cuatro soluciones** clásicas y argumentar por qué **ninguna es completa**.
6. Enumerar **ventajas y desventajas** de distance-vector con la nomenclatura de la cátedra.

> **Prerequisito del examen:** `2006-08 P3` (10 pts), `2008-12 Problema 2` (30 pts), `2010-02 Problema 2` (30 pts), `2010-07 Problema 2b`, `2007-02 P3` (10 pts).

PARTE 1 — TEORÍA

1.1 Tres maneras de que un router sepa a dónde mandar un paquete



El punto que vale la mitad de la nota conceptual: en (c) **no hay cálculo** (el *longest prefix match* busca en una tabla que alguien ya llenó); en (b) el cálculo lo hace **cada router por su cuenta** y el resultado se traduce a una tabla de forwarding. Cuando el enunciado pida "la tabla de forwarding", quiere esa **traducción** a `(Destino, Próximo Salto, Costo)`.

1.2 La ecuación de Bellman-Ford

```
c(x,v)  costo del enlace DIRECTO entre x y su vecino v.
        NO se envia: cada router lo tiene en su propia interfaz.
D_v(y)  lo que el vecino v me anuncio que le cuesta llegar a y.
D_x(y)  mi estimacion actual de la distancia a y.
```

$$D_x(y) = \min_{v \text{ en } V(x)} \{ c(x,v) + D_v(y) \}$$

Por qué la ecuación es correcta (principio de optimalidad)

Es lo que pide `2006-08 P3` cuando dice "**detalladamente**". Supongamos que el mejor camino de `x` a `y` es `x --(c1)--> n1 --(c2)--> n2 --> ... --> y`. `x` tiene que entregar el paquete a **alguien**, y ese alguien es `n1`. ¿Qué tiene que hacer `n1` desde `n1`?

> **El mejor camino de `n1` a `y`.** Porque si el tramo que le queda a `n1` **no fuera óptimo**, yo podría construir un camino mejor de `x` a `y`: tomaría mi enlace `x -> n1` (costo `c1`) y seguiría por ese camino suboptimal. Eso **contradice** que el mío era el mejor.

Eso es el **principio de optimalidad** (Bellman, 1957): *si el camino óptimo de `x` a `y` pasa por `v`, el tramo de `v` a `y` es él mismo óptimo desde `v`*. De ahí sale la ecuación: el camino óptimo **tiene que empezar** por algún vecino `v`; llegar a `v` cuesta `c(x,v)`; y de `v` a `y` cuesta como mínimo `D_v(y)`. Elegir `v` cuesta `c(x,v) + D_v(y)`, y el óptimo es el `min` sobre todos los vecinos. QED

Corolario: `D_x(y)` es no negativo y, como la recurrencia es sobre caminos **sin ciclos**, el algoritmo converge. **Justo lo que se rompe en §1.4**: el principio de optimalidad deja de ser aplicable cuando hay un bucle, porque el supuesto "sin ciclos" ya es falso.

1.3 La actualización: cuándo y qué se manda

Qué se manda: un VECTOR, no una topología. Eso es **todo** el mensaje:

```
┌──────────────────────────────────────────────┐
│  VECTOR DE DISTANCIAS de x  (= su tabla)     │
│   destino        distancia   próximo salto   │
│   192.168.1/24       0          local        │
│   192.168.2/24       2          R2           │
│   0.0.0.0/0          6          R2           │
└──────────────────────────────────────────────┘
```

Un LSA (clase siguiente) describe **un** enlace de quien lo emite; el vector describe **todas** las rutas de quien lo emite. De ahí el nombre "rumor": es una afirmación sobre el mundo entero, basada en información que el emisor ya está viejo.

```
PERIODICO:  un timer T (en RIP, 30 s). Al vencer, mando mi tabla completa
            por TODOS los enlaces.      + simple, robusto
                                        - una noticia mala tarda hasta T
DISPARADO POR CAMBIO (triggered update): mi tabla cambió, mando el vector
            AHORA.                      + la noticia buena viaja rápido
            - con la mala dispara MUCHAS veces seguidas, y ahí aparece el
              conteo a infinito. Por eso RIP limita los triggered updates
              a 1 cada 1 a 5 s.
```

El procedimiento exacto (solución oficial de `2006-08 P3`, citado):

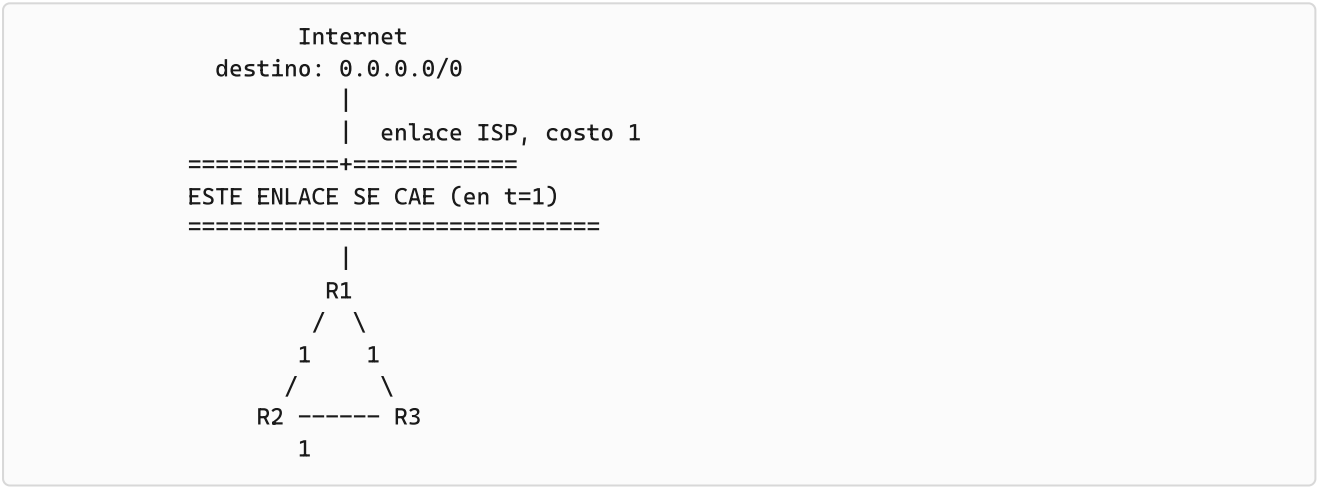
> Sea `T0` la tabla propia y `T1` la tabla recibida del router vecino `R`. Para cada ruta a un host `A` en `T1` **a través de** `R`: si **no existe** en `T0`, **agrego** la ruta hacia `A` por `R` incrementando la distancia; si

> **existe** en `T0` pero con una **métrica mayor**, la **sustituyo** por la de `R`; si **existe** la misma ruta, la marco como **"refrescada"**. Luego de recibidos updates por **todas** las interfaces, **borro** todas las rutas **"no refrescadas"**.

⚠ **El último punto responde a la pregunta clásica de DV: ¿cómo sabe `x` que un vecino dejó de existir?** No hay detección de caída de enlace en el mensaje: hay un **timeout por ruta**. Si una ruta no fue refrescada en esta ronda, el vecino que la anunciaba ya no existe y se borra. En RIP: **180 s**.

1.4 ⭐ El conteo a infinito — el diagrama más importante de la clase

Éste es **el** problema de distance-vector. Armemos la red mínima donde el conteo a infinito **ocurre de verdad**. La condición necesaria es que **el destino tenga una única boca de salida y que esa boca la crucen juntos dos routers**. Un triángulo simple NO sirve: el tercer router tiene siempre la verdad y el bucle se deshace en una ronda.



Enlaces del triángulo: `R1–R2 = 1`, `R1–R3 = 1`, `R2–R3 = 1`. Cada router tiene su LAN detrás, a costo 1. **Internet se alcanza únicamente saliendo por R1.**

t = 0 — estado estable

`D_R1(Internet) = 1 + 1 = 2`; `D_R2 = 1 + 2 = 3`; `D_R3 = 3`.

destino	R1	R2	R3
LAN1	**1** (local)	2 (vía R1)	2 (vía R1)
LAN2	2 (vía R2)	**1** (local)	2 (vía R2)
LAN3	2 (vía R3)	2 (vía R3)	**1** (local)
Internet	**2** (vía ISP)	3 (vía R1)	3 (vía R1)

t = 1 — R1 detecta la caída de su enlace al ISP

- El enlace de R1 con el ISP **no lo usa nadie más**: la **interfaz** lo reporta caído y R1 pone `D_R1(Internet) = ∞`.

- **R2 y R3 todavía no saben nada**: su enlace con R1 está perfecto y lo único que escucharon de R1 fue `Internet = 2`. Siguen con 3.

```
R1 -> Internet = INF          (¿y ahora qué? no tengo a quién preguntarle)
R2 -> Internet = 3 vía R1     (¡sigue usando un enlace que ya no sirve!)
R3 -> Internet = 3 vía R1
```

t = 2 — el descubrimiento: los tres se realimentan

Acá está la **semilla del bucle**. Cada uno escucha a sus vecinos y aplica la ecuación:

```
R1 recibe de R2: "Internet está a 3 por mí"    ->  c(R1,R2) + 3 = 1 + 3 = 4
R1 recibe de R3: "Internet está a 3 por mí"    ->  1 + 3 = 4
    ->  R1 adopta 4, vía R2      ¡el camino que le pasa R2 pasa por R1!

R2 recibe de R1: "Internet está a INF"         ->  1 + INF = INF  (invalida su 3)
R2 recibe de R3: "Internet está a 3 por R1"    ->  1 + 3 = 4
    ->  R2 adopta 4, vía R3      ¡el camino vuelve por R1!

R3 recibe de R1: "Internet está a INF"         ->  INF
R3 recibe de R2: "Internet está a 3 por R1"    ->  1 + 3 = 4
    ->  R3 adopta 4, vía R2      ¡y vuelve por R1 otra vez!
```

Leé el detalle importante: el `4` que R1 calcula sale de un rumor **de R2** que dice "llego a Internet en 3 por R1". O sea: **R2 le está contando a R1 un camino que pasa por R1 mismo.** Ese es el bucle, y por eso la métrica **sube** en vez de bajar a ∞.

t = 3, 4, 5, … — la cuenta ascendente

ronda	R1 oye de R2	R1 oye de R3	R1	R2	R3
2	1+4 = 5	1+4 = 5	**5**	**5**	**5**
3	1+5 = 6	1+5 = 6	**6**	**6**	**6**
4			**7**	**7**	**7**
k			*k+2*	*k+2*	*k+2*
14			**16**	**16**	**16**
15			**INF**	**INF**	**INF**

Interpretación. Durante ~14 rondas la red **cree** que tiene camino a Internet, y ese camino es un **círculo**: los paquetes dan vueltas hasta que el TTL los mate. En la práctica nunca se llega al ∞ matemático: en RIP el infinito es **16** y el *garbage collection* borra la ruta en la ronda 14. **Las filas de las LANs no se mueven nunca:** el conteo a infinito afecta **exactamente a la fila que se rompió**.

Por qué el problema persiste aunque los tres hagan todo bien

Los tres están **aplicando Bellman-Ford correctamente**: el error no es de implementación sino de la **ecuación**, porque el principio de optimalidad ya no aplica (el supuesto "sin ciclos" es falso). La red se

autoalimenta con un rumor falso. Por eso la solución no es "calcular mejor" sino **romper la realimentación**; y lo único que corta el círculo en el plano de datos es el **TTL** (ICMP tipo 11).

1.5 Las cuatro soluciones, y por qué ninguna es completa

(1) Split horizon

> **Regla:** no le anuncies a `X` lo que aprendiste **de** `X`.

Cada entrada marca **de quién** la aprendí; al generar el vector para `v` se omiten las entradas cuyo origen sea `v`.

(2) Split horizon con poison reverse (reverse envenenado)

> **Regla:** en vez de **silenciar**, anunciá **INF** ("inalcanzable") para lo que aprendiste de ese vecino.

El silencio es ambiguo: el vecino no sabe si te callaste porque no tenés nada o porque no te llegó. El poison reverse es una **afirmación explícita de falsedad**.

Los dos, aplicados a la red de §1.4 (modelo síncrono; `(*)` = "lo que yo tenía venía de vos, así que te enveneno INF"):

```
ronda  R1->R2  R1->R3  R2->R3  R3->R2  |  R1          R2          R3
──────────────────── SPLIT HORIZON ──────────────────────────────────
 1      INF      INF      3       3     |  INF         3           3
 2      INF      INF      4       4     |  4 vía R2    4 vía R3    4 vía R2
 2   silencio    4    silencio   4     |  (R1 se calló sobre R3; R3 sobre R2)
 3      INF      5      INF      5     |  5 vía R3    INF         5 vía R1
 4      INF      INF      INF      INF   |  INF         6 vía R1    INF
 5                                 |      INF         INF         INF
 ⇒ los tres en INF, en 5 rondas

──────────────────── POISON REVERSE ──────────────────────────────────
 1      INF      INF      3       3     |  INF         3           3
 2      INF      INF      INF (*)  INF (*)|  4           4           4
 3      INF (*)  4      INF      INF    |  5 vía R3    INF         5 vía R1
 4      5       INF (*)  INF      INF   |  INF         INF         INF
 ⇒ los tres en INF, en 4 rondas, y con el mensaje explícito
```

El caso real de `2010-07 P2b` (el ISP es la única salida de una red de 3 routers, R1--R2, R1--R3, R2--R3, con `D_R1(Internet) = 2`): al caerse el enlace R1--ISP, `t=1` deja a R1 en ∞ y R2/R3 siguen con 3 vía R1; en `t=2` los tres recalculan con lo que oyen y quedan en **4**; de `t=3` en adelante **suman 1 por ronda** hasta el 16 de RIP. Con split horizon + poison reverse: **3 rondas**. La oficial dice que R1 elige "a través de R1", lo cual está **mal**: *un router no puede usar su propia ruta como próximo salto*.

Limitación de los dos: atacan el rumor **de vuelta por el vecino inmediato**, no globalmente; con bucles de **tres o más** routers en malla, split horizon solo **no alcanza**. **`RFC 2080` dice textualmente que poison**

reverse es "el método preferido de operación", y es lo que usa RIP: una ronda menos y sin ambigüedad.

> **(3) Hold-down:** cuando una ruta cambia (sobre todo si empeora o pasa a ∞), **no aceptes ningún cambio** sobre ella durante `holddown` segundos. **Qué resuelve:** el *route flapping* corto: sin hold-down, `R2: 3 → INF`, `R1: INF → 5`, `R2: INF → 5` … un "ping-pong" de rutas. **Costo:** retrasa la recuperación de las buenas rutas (si el enlace vuelve en 1 s, con hold-down de 180 s la ruta queda muerta 3 minutos). Por eso nunca va solo.
>
> **(4) Triggered updates:** cuando tu tabla cambia, mandá el vector **en el acto**. **Qué resuelve:** la velocidad de las **buenas** noticias (sin ellos, una ruta nueva tarda hasta 30 s en existir en toda la red). **La paradoja:** *acelera la convergencia normal, pero también hace que el conteo a infinito sea rápido*. Por eso RIP los limita a uno cada **1 a 5 segundos**.

Por qué, con las cuatro juntas, el problema persiste

Porque las cuatro atacan **síntomas**, no la **causa**:

```
split horizon      atacan el RUMOR que vuelve por el vecino inmediato
poison reverse     (ataque local, no global)
hold-down          atacan la FLUCTUACION
triggered updates  atacan la VELOCIDAD (y la aceleran de mas)
      |
      └─> NINGUNA ataca la causa: que la metrica es un NUMERO y no un
          IDENTIFICADOR del camino. Dos caminos distintos con la misma
          metrica son INDISTINGUIBLES para Bellman-Ford. La informacion
          para romper el bucle (la ruta completa) NO ESTA en el mensaje.
```

La solución de fondo es la que usa BGP: mandar en el mensaje **la lista de por dónde pasó la ruta** (`AS_PATH`) y rechazarla si esa lista ya contiene a uno mismo. Ahí el bucle se detecta **exactamente**, no aproximadamente.

1.6 RIP — el DV que existió de verdad

`RIP` = **Routing Information Protocol**, primer protocolo de Internet (`RFC 1058`, 1988), derivado del `routed` de Berkeley. La versión que se estudia es **RIPv2** (`RFC 2453`, 1998).

Parámetro	Valor en RIPv2	Por qué
Métrica	**hops** (cada enlace vale 1)	la más simple de calcular
Infinito	**16**; máximo válido **15**	con tope, cualquier bucle se disuelve en ≤ 15 rondas
Split horizon / poison reverse	sí / sí (forma preferida)	§1.5
Triggered updates / hold-down	sí (throttle 1–5 s) / sí (~180 s)	
Timer periódico	30 s	
Timeout (ruta no refrescada)	180 s (6 × 30)	así se detecta que un vecino murió
Garbage collection	120 s tras el timeout	anuncia 16 para que los vecinos también borren
Transporte	**UDP 520**	sin conexión, best effort
Mensaje	Request (1) / Response (2); RTE de 20 bytes; máx. 25 RTE (500 B)	

RIPng: la versión para IPv6 — `RFC 2080` (enero 1997, Proposed Standard). Es **el mínimo cambio** de RIPv2: prefijo de **128 bits**; métrica **1..15** y **16 = ∞**; **split horizon con poison reverse** ("método preferido"); **timers idénticos** (30 s / 180 s / 120 s) más un **offset aleatorio de ±15 s** para que los routers no se sincronicen; **UDP 521** y **multicast `FF02::9`**; campo **route tag** para separar rutas internas de externas; y el **next hop va en un RTE especial** (métrica `0xFF`) que aplica a todas las entradas siguientes.

⚠️ Actualización a 2026

Sigue igual, de memoria: la ecuación de Bellman-Ford y el principio de optimalidad; el conteo a infinito y las cuatro soluciones; **RIPv2** con hops, tope 15 e infinito 16; y que **RIP es Internet Standard pero está muerto de facto**.

Cambió o hay que matizar: - **No existe un "RIP v3" ni un reemplazo de RIP.** Lo que cambió es que **nadie lo despliega**: sobrevive en laboratorios y campus. Su lugar lo tomó **OSPF** (interior) y **BGP** (exterior). - Lo que reemplazó la lógica de "elegir el camino más corto" en los operadores es el **Segment Routing** (`RFC 8402`, `RFC 8660` SR-MPLS, `RFC 8986` SRv6): *source routing* con SIDs y **estado sólo en el borde de ingreso**. En datacenter, **EVPN (`RFC 7432`) + VXLAN (`RFC 7348`)** sobre UDP/4789. - ⚠️ **Trampa:** si preguntan "el RIP de 2026", la respuesta es **RIPv2, `RFC 2453`, sin cambios**.

1.7 Ventajas y desventajas — la respuesta de `2006-08 P3`

Las TRES ventajas: (1) Simplicidad: es un `loop`, un `min` y una tabla. **(2) Poco estado:** cada router guarda **su** tabla de `N` entradas y **no conoce la topología**. **(3) Funciona en topologías donde otros no:** sólo necesita vecinos que le hablen (el único IGP que sobrevive en NBMA).

Las CUATRO desventajas: (1) Convergencia lenta: hasta `D` rondas en una red de `D` saltos. **(2) "Rumor" e inconsistencia transitoria:** la tabla de `x` no es el camino de nadie, es una **combinación** de lo que dijo

cada vecino en el último tick; puede haber bucles, caminos imposibles y *black holes*. **(3) El conteo a infinito (§1.4). (4) Escala:** `N` routers × `N` destinos = `N²` **entradas circulando por ronda**.

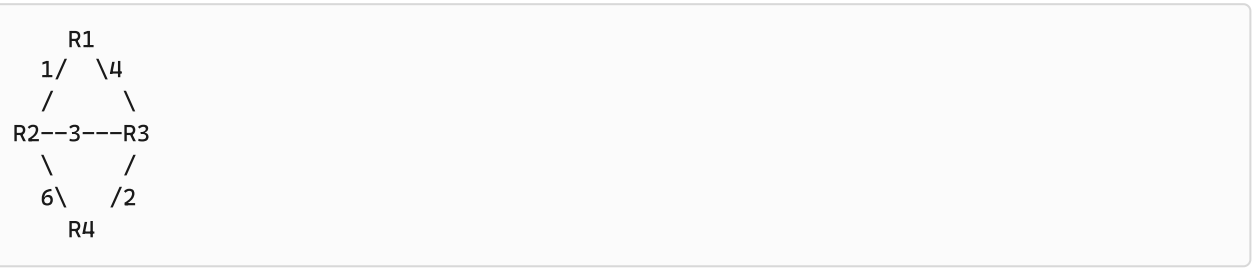
Cada ronda cada router manda su tabla COMPLETA a cada vecino: circulan `N x N` entradas. Con 1 000 routers son 10^6 entradas por ronda = 4 MB por router cada 30 s = **~133 kB/s sostenidos sólo de ripo, en cada router**. Ese es el motivo del tope de 15 hops.

	Distance-Vector	Link-State (OSPF)
Qué conoce cada nodo	sólo lo que le dicen sus vecinos	la topología **completa**
Qué se envía	el vector de distancias (`N` entradas)	un LSA por enlace (pocos bytes)
Algoritmo	Bellman-Ford	Dijkstra
Convergencia	**lenta**, con conteo a infinito	**rápida**, sin conteo a infinito
Memoria por router	`O(N)`	`O(V+E)` del área
Control por ronda	`O(N²)`	`O(N)`
Tope práctico	15 hops	miles de routers

Una distinción que separa a los que entendieron: el *forwarding* es un problema de **tiempo real** (se ejecuta por paquete, error = paquete perdido); el **cálculo de enrutamiento** es de **tiempo diferido** (cada 30 s, no hay deadline, error = la red anda mal). Por eso los protocolos de enrutamiento pueden ser lentos: **no están en el camino crítico del paquete**.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — 4 routers con costos distintos, convergencia en 3 rondas



Costos: `R1–R2 = 1`, `R2–R3 = 3`, `R3–R4 = 2`, `R1–R3 = 4`, `R2–R4 = 6`. Los routers arrancan **sincrónicamente** con la tabla vacía.

Paso 0 — sólo vecinos directos. Cada router conoce `D_x(x) = 0` y `c(x,v)` de sus vecinos (los costos están en sus interfaces, nadie tiene que decírselos).

	R1	R2	R3	R4
R1	**0**	**1**	**4**	INF
R2	**1**	**0**	**3**	**6**
R3	**4**	**3**	**0**	**2**
R4	INF	**6**	**2**	**0**

Ronda 1:

```
D_R1(R4) = min{ 1 + D_R2(R4), 4 + D_R3(R4) }     = min{ 1+6, 4+2 }     = 6  vía R3
D_R2(R4) = min{ 1 + D_R1(R4), 3 + D_R3(R4), 6 } = min{ 1+INF, 3+2, 6+0 } = 5  vía R3
D_R4(R1) = min{ 2 + D_R3(R1), 6 + D_R2(R1) }     = min{ 2+4, 6+1 }     = 6  vía R3
D_R4(R2) = min{ 2 + D_R3(R2), 6 + D_R2(R2) }     = min{ 2+3, 6+0 }     = 5  vía R3
```

	R1	R2	R3	R4
R1	0	1	**1**	**6**
R2	1	0	3	**5**
R3	4	3	0	2
R4	**6**	**5**	2	0

Ronda 2:

```
D_R2(R3) = min{ 1 + D_R1(R3), 3 + D_R3(R3), 6 + D_R4(R3) }
         = min{ 1+1, 3+0, 6+2 } = 2  vía R1     <-- CAMBIÓ
```

Interpretación: R2 descubrió que ir a R3 pasando por R1 (costo `1+1 = 2`) es **más barato** que el enlace directo de costo 3. Acá está el corazón de DV: la métrica no la da el enlace, la da **todo** el camino.

	R1	R2	R3	R4
R1	0	1	1	6
R2	1	0	**2**	5
R3	4	3	0	2
R4	6	5	2	0

Los demás no cambian: `D_R1(R4) = min{1+5, 4+2} = 6` (empate, vía R3); `D_R3(R1) = min{3+1, 4+0, 2+6} = 4` (empate, directo); `D_R4(R1) = 6`; `D_R4(R2) = 5`.

Ronda 3 — convergió: `D_R2(R3) = min{1+1, 3+0, 6+2} = 2`; `D_R1(R4) = min{1+5, 4+2} = 6`; `D_R3(R1) = min{3+1, 4+0, 2+6} = 4`; `D_R4(R1) = min{2+4, 6+1} = 6`; `D_R4(R2) = min{2+3, 6+0} = 5`. **Ninguna celda cambió ⇒ convergió en 3 rondas.**

Caminos óptimos: R1→R2 directo (1) · R1→R3 **vía R2** (1+1 = 2, ¡mejor que el directo de 4!) · R1→R4 **vía R3** (4+2 = 6) · R2→R3 **vía R1** (1+1 = 2) · R2→R4 **vía R3** (3+2 = 5) · R3→R1 directo (empate con R3→R2→R1: 4 = 3+1) · R4→R2 **vía R3** (2+3 = 5).

⚠️ **Errores típicos: olvidar el `+ c(x,v)`** (el error nº 1); **confundir el paso 0** ("vacía" no es "todo ∞"); **no decir en qué ronda convergió**; y en los empates, elegí el directo y **decilo** — la cátedra en `2010-07 P2b` dice "*ante caminos de igual distancia se elige uno al azar*".

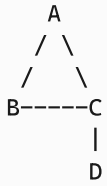
PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2008-12, Problema 2 (30 puntos) — distance-vector en una red de 4 nodos

> **Fuente:** Examen `eirc0812` / `sirc0812` — 20 de diciembre de 2008. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0812.pdf
>
> **Texto del examen:** *Se considera una red de cuatro nodos A, B, C y D con la topología que se muestra en la figura; los enlaces tienen costo unitario. Esta red ejecuta un protocolo de enrutamiento genérico de la familia "vector-distancia" (distance-vector).* **a)** *...los nodos se inician todos en el mismo estado con sus tablas de distancias vacías; deduzca las sucesivas tablas de distancias de A, B, C y D hasta llegar al estado estable.* **b)** *Escriba la tabla de forwarding de cada nodo en el formato Destino, Próximo Salto, Costo.* **c)** *Explique qué sucede con las tablas de distancias si se apaga el nodo D.*

La topología — *adaptamos la figura a texto*

La figura del PDF es una imagen; los valores de la tabla de abajo son los del examen y se deducen de las tablas que la cátedra publica como solución. La red es **un triángulo A-B-C con D colgando de C**:



enlace	A	B	C	D
A	—	1	1	INF
B	1	—	1	INF
C	1	1	—	**1**
D	INF	INF	**1**	—

Cómo lo verifiqué: la tabla de distancias que `sirc0812` publica como estado estable es `A(0,1,1,2) B(1,0,1,2) C(1,1,0,1) D(2,2,1,0)`, y eso **sólo** es consistente con `A–B=1`, `A–C=1`, `B–C=1`, `C–D=1` y **sin** enlace directo `A–D` ni `B–D`: si hubiera `A–D=1`, la distancia A→D sería 1, no 2.

Parte a) — las sucesivas tablas de distancias

Ronda 0 (arranque con tablas vacías ⇒ sólo vecinos directos). ⚠️ "Tablas vacías" significa que **no hay ninguna ruta aprendida de nadie**. Lo que el router sabe de entrada es `D_x(x) = 0` y `c(x,v)` de sus vecinos, porque los costos de sus propios enlaces están en sus interfaces. La cátedra rotula esta ronda `t=0`, y ahí las tablas son, de hecho, la **fila de costos** de la matriz de adyacencia.

desde A	A	B	C	D
A	0	1	1	INF

desde B	A	B	C	D
B	1	0	1	INF

desde C	A	B	C	D
C	1	1	0	1

desde D	A	B	C	D
D	INF	INF	1	0

Ronda 1 — primera ronda de intercambio. Para el destino `D`:

```
A:  D_A(D) = min{ 0+D_A(D), 1+D_B(D), 1+D_C(D), INF+D_D(D) } = min{INF, 1+INF, 1+1, INF} = 2
B:  D_B(D) = min{ 1+D_A(D), 0+D_B(D), 1+D_C(D), INF+D_D(D) } = min{1+INF, INF,  1+1, INF } =
C:  D_C(D) = min{ 1+D_A(D), 1+D_B(D), 0+D_C(D), 1+D_D(D) } = min{INF, INF,  0+1, 1+0 } = 1
D:  D_D(A) = min{ INF, INF, 1 + D_C(A), 0 + D_D(A) }            = 1 + 1 = 2              v
   D_D(B) = min{ INF, INF, 1 + D_C(B), 0 + D_D(B) }            = 1 + 1 = 2              ví
```

desde A	A	B	C	D
	0	1	1	2

desde B	A	B	C	D
	1	0	1	2

desde C	A	B	C	D
	1	1	0	1
= ESTADO ESTABLE				

desde D	A	B	C	D
	2	2	1	0
= ESTADO ESTABLE				

Interpretación — la observación clave del examen: una sola ronda bastó, porque `C` es el **único cuello de botella** hacia `D` y todo lo que había que aprender de `D` cabía en su mensaje. La red tiene **diámetro de información 1** hacia `D`: es lo que se llama **convergencia trivial**.

Ronda 2 — se verifica que no cambia nada: `A: min{INF, 1+2, 1+1, INF} = 2` ✓ · `B: min{1+INF, INF, 1+1, INF} = 2` ✓ · `C: min{1+2, 1+2, 0+1, 1+0} = 1` ✓ · `D: min{INF, INF, 1+1, 0} = 2` ✓. ⇒ **El estado estable se alcanza en la ronda 1.**

Parte b) — las tablas de forwarding

A	Próx. salto	Costo		**B**	Próx. salto	Costo
A	— (local)	0		A	— (local)	0
B	B	1		B	— (local)	0
C	C	1		C	C	1
D	C	2		D	C	2

C	Próx. salto	Costo		D	Próx. salto	Costo
A	A	1		A	C	2
B	B	1		B	C	2
C	— (local)	0		C	C	1
D	D	1		D	— (local)	0

Comparación con la oficial. `sirc0812` publica estas cuatro tablas (con la extracción descentrada) y **coinciden con las mías**: para `A`, `(B,B,1) (C,C,1) (D,C,2)`; para `D`, `(A,C,2) (B,C,2) (C,C,1)`. Lo que la oficial no publica son las completas de `B` y `C`; las derivé igual. ⚠ **El destino "A" de la tabla de A tiene próximo salto = local, costo 0**: no es "A→A", es "esto lo despierto yo".

Parte c) — Qué pasa si se apaga D

Solución oficial (citada):

> *"Si se apaga el nodo D se produce un fenómeno de 'conteo a infinito' en los nodos A y B. Esto se debe a que cada uno dice saber llegar a C con cierta distancia (inicialmente 1) y en cada intercambio de tablas se va incrementando en cada nodo (A y B), potencialmente a infinito."*

Se cae el nodo `D`, o lo que es lo mismo, el enlace `C–D` (único que `D` tenía):

```
ronda 2
C:  D_C(D) = min{ 1+D_A(D), 1+D_B(D), 0+INF, INF } = min{1+2, 1+2, INF, INF} = 3  (vía A o B
A:  D_A(D) = min{ 1 + D_B(D), 1 + D_C(D) }       = min{1+2, 1+3} = 3  <-- CAMBIÓ
B:  D_B(D) = min{ 1 + D_A(D), 1 + D_C(D) }       = min{1+3, 1+3} = 3  <-- CAMBIÓ

ronda 3 :  A: min{1+3, 1+3} = 4 ;  B: 4 ;  C: min{1+3, 1+3, INF, INF} = 4
ronda 4 :  A: 5 ;  B: 5 ;  C: 5
ronda k :  los tres suben 1 por ronda
```

ronda	`D_A(D)`	`D_B(D)`	`D_C(D)`
0	2	2	1
1	2	2	**INF**
2	**3**	**3**	3
3	**4**	**4**	4
4	**5**	**5**	5
k	*k+1*	*k+1*	*k+1*
15	**INF**	**INF**	**INF**

⚠️ **El error conceptual más caro de esta clase, y donde muchas soluciones se caen:** A **no** se queda con su `2` viejo. DV **recalcula desde cero** cada ronda sobre lo que dicen los vecinos: `min{1+2, 1+3} = 3`. No es "monótono decreciente".

Interpretación. Durante 15 rondas los tres routers **creen** que llegan a la red de D por un camino que **ya no existe**: los paquetes que salgan con esas tablas se pierden. En RIP se corta solo en la ronda 15, cuando la métrica llega a 16 y se borra.

Lo que lo arregla: poison reverse. `C` es el único que tiene la verdad, así que en cuanto envenena `D = 16` para todos, `A` y `B` bajan directo a ∞ en **una** ronda (todo en `t=2`). ⇒ **15 rondas de convergencia mala contra 1 de convergencia correcta.** Ese es el argumento si el enunciado pide "cómo lo mejoraría".

Nota: el mismo problema en `2010-02` (`sirc1002`), y dos desvíos del PDF oficial

La topología es idéntica y las tablas son las de la parte a). Lo valioso son dos desvíos del PDF oficial. **(1) La numeración de las rondas:** el bloque rotulado `t=1` para el nodo `A` dice `0 1 INF 1`, que es **exactamente la tabla inicial**: **el PDF duplica la tabla inicial** en el rótulo `t=1` (lo mismo en `sirc0812`). La lectura correcta es `t=0` inicial, `t=1` primera ronda (`A(0,1,1,2) B(1,0,1,2) C(1,1,0,1) D(2,2,1,0)`), `t=2` sin cambios. **Una ronda alcanza**, porque para el destino `D` toda la información cabe en el mensaje de `C` (que anuncia `D=1`). **(2) Un error de tipeo:** la tabla `t=2` de `A` dice `0 1 2 1`, o sea `D_A(D) = 1`, lo cual **no puede ser** (implicaría un enlace directo `A–D` que no existe; si existiera, la tabla estable de `sirc0812` tendría `A→D = 1`, y tiene 2). La fila correcta es `0 1 1 2`. ⚠ **Cuando el PDF y la matemática no cierran, reconstruí la topología desde los datos de otras partes, declarás el supuesto y escribilo.**

La solución oficial de la parte b) dice: "*el nodo C detecta que la ruta más corta a D es inaccesible y por lo tanto elige la segunda más corta que es a través de B con distancia total 3*" — verificado, `1 + D_B(D) = 1 + 2 = 3`, **la cuenta está bien escrita**; pero "*luego B aumenta la suya a 4*" usa un modelo **asíncrono**, cuando las tablas del enunciado son **síncronas** (`D_B(D) = min{1+2, 1+3} = 3` en la primera ronda).

3.4 Examen 2006-08, Pregunta 3 (10 puntos) — funcionamiento, ventajas y desventajas

> **Fuente:** Examen `sirc0608` — 5 de agosto de 2006. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0608.pdf **Texto del examen:** *Describa detalladamente el funcionamiento de un protocolo de enrutamiento de tipo DISTANCE-VECTOR (VECTOR-DISTANCIA). Mencione posibles ventajas y desventajas del mismo.*

(a) Funcionamiento (≈5 puntos), en cinco pasos. (1) Modelo de tabla: un **vector de distancias** por destino, con la distancia mínima y el **primer salto**. **(2) Inicialización:** arranca "vacía", o sea sólo conoce `D_x(x) = 0` y `c(x,v)` de sus vecinos directos. **(3) Timer:** un timer `T` (en RIP, 30 s); al vencer, envía su tabla a **todos** sus vecinos. **(4) Actualización:** al recibir la tabla de un vecino `R`, la **compara con la propia** y la actualiza con Bellman-Ford. **(5) Refresco:** toda ruta no refrescada en la ronda se **borra**, y así se detecta que un vecino dejó de anunciar.

(b) Ventajas (≈2,5 pts) y (c) desventajas (≈2,5 pts): las tres y las cuatro de §1.7.

Solución oficial (citada): "*Conteo a Infinito. Las malas noticias se propagan lento. Cuando un enlace cae, hay un gran número de iteraciones hasta que todos los nodos se dan cuenta que hay un problema.*" ⚠ "Las

malas noticias se propagan lento" y "el conteo a infinito" son **el mismo problema desde dos ángulos** (síntoma vs. mecanismo).

Bonus del Problema 1 de ese mismo examen (6 routers): la columna "**Costo en dicho enlace**" es el costo del **primer salto**, no la distancia — la distancia es la **suma** de todos los saltos. Y cada router tiene **dos entradas** por LAN remota ⇒ **ECMP** (*Equal-Cost Multi-Path*). Y el `TTL` es lo único que corta un bucle en el plano de datos: al llegar a 0 manda un ICMP tipo 11.

PARTE 4 — CIERRE

📌 Resumen

1. **Distance-vector**: cada router conoce **la distancia a cada destino** y **el primer salto**. No conoce la topología.
2. Por el enlace viaja un **vector de distancias**, no la topología: por eso se lo llama "rumor".
3. La ecuación es `D_x(y) = min_v{ c(x,v) + D_v(y) }`, correcta por el **principio de optimalidad**.
4. La actualización es **periódica** (30 s en RIP) o **disparada por cambio** (triggered update); las rutas no refrescadas se borran.
5. **Conteo a infinito**: un router vuelve a aceptar un rumor que él mismo originó, y la métrica **sube 1 por ronda**. **DV recalcula desde cero** cada ronda: no es monótono decreciente.
6. Las **4 soluciones**: split horizon, poison reverse, hold-down, triggered updates. **Ninguna es completa**: la métrica es un número, no un identificador de camino.
7. **RIP (`RFC 2453`)**: hops, tope **15**, infinito **16**, UDP 520. **RIPng (`RFC 2080`)**: UDP 521, `FF02::9`, 128 bits.
8. **Ventajas**: simple, poca memoria, topologías raras. **Desventajas**: convergencia lenta, rumor, conteo a infinito, escala `O(N²)`.
9. La **solución de fondo** es **path-vector** (BGP): mandar la lista de por dónde pasó la ruta y rechazarla si uno ya está en ella.
10. **Forwarding ≠ enrutamiento**: el forwarding consulta la tabla en tiempo real; el cálculo no tiene deadline.

✅ Autoevaluación

1. Escribí la ecuación de Bellman-Ford y explicá por qué el camino óptimo de `x` a `y` empieza por el vecino que minimiza `c(x,v) + D_v(y)`.

▶ Respuesta

2. En la topología del §1.4, ¿en qué ronda exacta R1 pasa de ∞ a 4?

▶ Respuesta

3. ¿Por qué poison reverse converge más rápido que split horizon solo?

▶ Respuesta

4. Un router tiene dos rutas al mismo destino de costo 3, aprendidas de R2 y de R3. ¿Qué pasa con split horizon?

▶ Respuesta

5. ¿Por qué el enunciado dice "tablas vacías" y por qué la primera tabla no es todo ∞?

▶ Respuesta

6. ¿Cuánto tarda RIP en suprimir una ruta a 15 hops? Explicá el mecanismo.

▶ Respuesta

7. ¿Qué tiene de malo decir que DV es "$O(N^2)$"?

▶ Respuesta

8. Cuando R1 tiene `Internet = 4 vía R2`, R2 `4 vía R3` y R3 `4 vía R2`, ¿qué le pasa a un paquete real que sale de R1?

▶ Respuesta

🔗 Referencias

- **`eirc0608` / `sirc0608`** — P3 (10 pts) · Problema 1 (30 pts: 6 routers, TTL, traceroute). Ojo: "**Costo en dicho enlace**" es el costo del **primer salto**, no la distancia; y hay **dos entradas** por LAN remota ⇒ **ECMP**.
- **`eirc0702`** — P3 (10 pts: conteo a infinito y una solución). *Sin solución oficial publicada.*
- **`eirc0812` / `sirc0812`** — Problema 2 (30 pts) · **`eirc1002` / `sirc1002`** — Problema 2 (misma red) · **`sirc1007`** — Problema 2b
- **`sirc0008`** — Problema 1: **aprendizaje reverso** (*backward learning*). Es DV sin métricas: al recibir un datagrama de origen `H` por la interfaz `origen`, `x` aprende "para llegar a `H`, salí por `origen`". Elige con **papa caliente** (métrica = `tamaño_cola`) y resuelve los cambios de topología con **envejecimiento**, que es **exactamente el timeout de RIP**.
- **Tanenbaum & Wetherall**, *Redes de computadores** 5ª ed., cap. 5 §5.6 "Interior Gateway Protocols"
- **RFC 2453** — RIP v2 · **RFC 1058** — RIP · **RFC 2080** — RIPng · **RFC 8402 / 8660 / 8986** — Segment Routing · **RFC 7432 / 7348** — EVPN / VXLAN
- **Clase siguiente:** 33 — Enrutamiento link-state, Dijkstra y OSPF